

Project 3: MDPs and ML

01:198:440

You are a brave little bot.

You have now mapped out your ship, and you have located the rat. Unfortunately, now the rat is on the run.

For this project, you may take the map as known to the bot, and the location of the rat as known at all times.

You and the rat alternate moves. Every time you move, if you enter the rat's cell, the rat will be teleported safely off the ship. Every time the rat moves, the rat will choose a neighboring cell to run to (uniformly at random from available neighbors). If the rat enters your cell, the rat is safely teleported off the ship.

Note, you and the rat technically move at the same speed, but the rat is not trying to flee directly away from you - this gives you an advantage.

Your goal is to remove the rat as efficiently as possible (as few moves as is necessary). Fortunately for you, this is no longer a question of trying to come up with creative bot strategies. We can compute the optimal strategy.

Let $T(b_x, b_y, r_x, r_y)$ be the *minimal* expected number of moves to remove the rat, when the bot starts at cell (b_x, b_y) in the ship, and the rat starts at cell (r_x, r_y) .

Note, you can think of T as a $D \times D \times D \times D$ size array, where D is the size of the ship.

Questions to Answer:

- For what values of (b_x, b_y) and (r_x, r_y) is T easy to compute? And I mean just immediate.
- Note, if we store T as a $D \times D \times D \times D$ array, we have to worry about coordinates of blocked cells where the bot and rat can't actually be. What should T be here?
- For $(b_x, b_y), (r_x, r_y)$ that are valid, mathematically express $T(b_x, b_y, r_x, r_y)$ in terms of the actions the bot can take, and the 'next' configurations the bot and rat might be in.
- Describe (clearly, in detail) an algorithm to compute the value of T for every possible bot/rat configuration for a given ship.
- Generate a ship, compute T for that ship, and show the bot/rat configurations that have maximum T .
- Is the bot always going to catch the rat?
- Once you calculate T , you should be able to determine the optimal action for the bot to take in every configuration. How?
- How often is the bot's optimal action just to move along a shortest path to the rat? As a percentage of total configurations. (Average your results over multiple ships.)

Once you have the ability to generate these T arrays for a given ship, I'd like to use these as a basis for doing a little bit of machine learning.

Questions to Answer:

- If there are N open cells in a ship, how many possible configurations can the rat and bot be in?

We can think of a computed T function as a set of data points: for each input configuration of bot and rat, we have the corresponding output - the expected number of moves needed to catch the rat.

To complete this project, I would like you to train a machine learning model on this data, to be able to predict the expected number of moves needed to catch the rat.

Questions to Answer:

- How should you represent your input?
- How should you represent your output?
- Describe your model space in detail. Note, you are making decisions when you specify a model space - be clear about the choices you are making.

Note, if your initial model does not produce good results - it is expected that you experiment with others, try new things, until you find something better / something that works.

- How are you judging the quality of your model?
- What training algorithm are you using to find the best model? Be clear about any decisions you are making.

Results to Present:

- Show that your model is learning, i.e., is able to perform better predictions as it trains. (What should you graph to show this?)
- Show that your model is doing more than just memorizing the individual data points. How? (What should you graph to show this?)
- After you are able to get a model working on data from one ship, try to build and train a model capable of working across multiple ships. What has to change? Be clear and specific about your approach and your results.